APPLICATION #2: SELF-BOOTSTRAP TOOLCHAIN

12 Patent Drawings -- ASCII Reference Diagrams

for tracing with Draw.io / Visio / PowerPoint

==

FIG. 1 -- Toolchain Subsystem Block Diagram

==

```
  hl-bootstrap.hl (4,572 lines, 208 functions)
  +==================================================================+
  |                                                            |
  |  +----------+   +----------+   +----------+   +-----------+    |
  |  | S1 (210) |   | S2 (220) |   | S3 (230) |   | S4 (240)  |   |
  |  | Spatial  |   | Lexical  |   | Syntactic|   | Interp /  |   |
  |  | Coord    |   | Analyzer |   | Analyzer |   | VM        |   |
  |  | Engine   |   |          |   |          |   |           |   |
  |  +----+-----+   +----+-----+   +-----+----+   +-----+-----+   |
  |       |              |              |              |          |
  |       |              |    (interpret path)           |          |
  |       |              +-------->+----->+------->+------+          |
  |       |              |         |              |               |
  |       +----> S3 -----+         |              |               |
  |       |              |         |              |               |
  |       +----> S5 -----+    (compile path)       |               |
  |             |        |                        |               |
  |  +-----------+--+   +-+----------+   +---------+--------+      |
  |  | S5 (250)     |   | S6 (260)   |   | S7 (270)       |      |
  |  | x86_64       |   | Bootable   |   | Command-Line    |      |
  |  | Machine Code |   | Image      |   | Interface       |      |
  |  | Encoder      |   | Builder    |   |                |      |
  |  +--------------+   +------------+   +-----------------+      |
  |                                                            |
  +==================================================================+
```

Data flow:

Interpret: S7 --> S2 --> S3 --> S4 --> (result)

Compile: S7 --> S2 --> S3 --> S5 --> S6 --> (bootable image)

S1 connected to S3 and S5 (coordinate primitives)

FIG. 2 -- Self-Bootstrap Cycle

==

```
                  hl-bootstrap.hl
                  (Source Code)
```

```
             |
             | compiled by
             v
+----------------------------+
|  Stage-0 Image            |
|  hicos-hl.img             |
|  160,768 bytes            |
+-------------+-------------+
             |
             | boots and runs
             v
+----------------------------+
|  Running Toolchain        |
|  (on QEMU / bare metal)   |
+-------------+-------------+
             |
             | reads source, compiles
             v
+----------------------------+
|  New Image                |
|  (functionally identical) |
+-------------+-------------+
             |
             | replaces Stage-0
             v
        (back to top)

+------------------------------+
| Self-Bootstrap Closed Loop  |
+------------------------------+
```

FIG. 3 -- Lexical Analysis Pipeline
==

```
+---------------------+
| Source Text        |
| (character stream)  |
+--------+-----------+
         |
         v
+--------+-----------+
| Character-by-char  |
| scan loop          |
+--------+-----------+
         |
```

```
    +-------+-------+-------+-------+--------+
    |       |       |       |       |        |
    v       v       v       v       v        v
  +------+ +------+ +------+ +------+ +------+ +------+
  |digit?| |quote?| |letter| |oper  | | //   | |other |
  |      | |      | |  ?   | |char? | |      | |      |
  +--+---+ +--+---+ +--+---+ +--+---+ +--+---+ +--+---+
     |        |        |        |        |        |
     v        v        v        v        v        v
  +------+ +------+ +------+ +------+ +------+ +------+
  |TT_NUM| |TT_STR| |TT_   | |TT_OP| |skip  | |error |
  |      | |      | |IDENT | |      | |      | |      |
  +------+ +------+ +------+ +------+ +------+ +------+
     |        |        |        |
     +--------+--------+--------+
              |
              v
   +---------+---------+
   | Token Stream      |
   | [type, value] pairs|
   +--------------------+
```

FIG. 4 -- Abstract Syntax Tree Structure
==

```
                       Program
                    (stmt list)
                   /     |     \
                  /      |      \
                 v       v       v
              FnDef     Let    While
             /  |  \    / \    /   \
            v   v   v  v   v  v     v
          name params body name expr cond body
           |    |     |
           v    v     v
         "add" [a,b]  Return
                       |
                       v
                     BinOp
                    /  |  \
                   v   v   v
                  "+"  a    b
```

Node types:

```
    FnDef    -> [name, params, body]
    If       -> [cond, then, else]
    While    -> [cond, body]
    Let      -> [name, expr]
    BinOp    -> [op, left, right]
    FnCall   -> [name, args]
  * Quadrant -> [name, body]  <-- UNIQUE to H-L

  Example: fn add(a, b) { return a + b; }
```

FIG. 5 -- Interpreter/VM Block Diagram

```
================================================================

    +-----------------+
    | AST Input      |
    +--------+--------+
             |
             v
    +--------+-----------------+
    |  Tree-Walk Evaluator     |
    |                        |
    |   evaluate(node, env)     |
    |   dispatch by node type   |
    +---+-------+-------+------+
        |       |       |
        v       v       v
  +----------+ +----------+ +-------------------+
  | Value    | | Environ  | | Function         |
  | Stack    | | Chain    | | Closures         |
  | (1024)   | | (scopes) | |                  |
  +----------+ +-----+----+ +-------------------+
                     |
                     v
           +-----------+      +-------------------+
           | Quadrant  |      | Built-in         |
           | Scope     |      | Functions        |
           | Resolution|      | (print, to_string |
           +-----------+      |  len, push, ...)  |
                             +-------------------+
                     |
                     v
           +-------------+
           | Execution   |
           | Result      |
           +-------------+
```

FIG. 6 -- x86_64 Machine Code Encoding

```
================================================================

  REX Prefix byte structure:
  +------+------+------+------+------+------+------+------+
  | 0    | 1    | 0    | 0    | W    | R    | X    | B    |
  +------+------+------+------+------+------+------+------+
    fixed prefix (0100)        W=64bit R=reg  X=SIB  B=rm

  Register encoding:
  +-----+-----+-----+-----+-----+-----+-----+-----+
  | RAX | RCX | RDX | RBX | RSP | RBP | RSI | RDI |
  |  0  |  1  |  2  |  3  |  4  |  5  |  6  |  7  |
  +-----+-----+-----+-----+-----+-----+-----+-----+

  Example encodings:
  +-----------------------+--------------------------+
  | Instruction           | Bytes                    |
  +-----------------------+--------------------------+
  | MOV RAX, imm64        | [0x48] [0xB8+r] [imm64]  |
  | PUSH r64              | [0x50+r]                 |
  | RET                   | [0xC3]                   |
  | SYSCALL               | [0x0F] [0x05]            |
  | IRETQ                 | [0x48] [0xCF]            |
  +-----------------------+--------------------------+

  Byte Buffer abstraction:
  +-------------------------------------------------+
  | buf_emit(byte)  | buf_emit16(u16) | buf_emit32() |
  | buf_emit64(u64) | buf_pos()       | buf_patch()  |
  +-------------------------------------------------+
```

FIG. 7 -- Multi-Stage Compilation Pipeline

```
================================================================

  +--------+    +-------+    +--------+    +----------+
  | Source |--->| Lexer |--->| Parser |--->| H-IR     |
  | (.hl)  |    | (S2)  |    | (S3)   |    | Lowering |
  +--------+    +-------+    +--------+    +-----+----+
                                                 |
                                                 v
                                           +-----------+
                                           | H-IR      |
```

```
                                | 37 opcodes|
                                +-----+-----+
                                      |
                                      v
  +--------+    +---------+    +----------+    +----------+
  |Bootable|<---| x86_64  |<---| Register |<---| Optimize |
  | Image  |    | Encoder |    | Alloc    |    |          |
  | (S6)   |    | (S5)    |    | Linear   |    | ConstFold|
  +--------+    +---------+    | Scan     |    | DCE      |
                               | 14 GP    |    | Strength |
                               | regs     |    | Reduce   |
                               +----------+    +----------+
```

FIG. 8 -- Boot Image Disk Layout

==

```
  Byte offset:
  0             511  512                           160,767
  +---------------+----+-----------------------------+---------+
  |  Stage 1:     |    |  Stage 2:                   | Stage 3:|
  |  MBR          |    |  CPU Mode Transition        | Kernel  |
  |               |    |                             | Binary  |
  |  512 bytes    |    |  Real -> Protected -> Long  | 118 H-L |
  |               |    |                             | modules |
  +---------------+----+-----------------------------+---------+
  Sector 0            Sectors 1 - N                 N+1 - 313

  Total: 160,768 bytes (314 sectors x 512 bytes)

  Sector 0 detail:
  +-----------------------------------------+----+----+
  | Boot code (510 bytes)                   |0x55|0xAA|
  +-----------------------------------------+----+----+
  byte 0                                 510   511
```

FIG. 9 -- CPU Mode Transition Sequence

==

```
  +---------------------+
  | 16-bit Real Mode    |  <-- BIOS entry
  | (entry point)       |
  +----------+----------+
             |
             | CLI (disable interrupts)
```

```
          | LGDT (load GDT)
          | Set CR0.PE = 1
          | Far JMP selector 8
          v
+---------------------+
| 32-bit Protected    |
| Mode                |
+----------+----------+
           |
           | Copy kernel to 0x100000 (REP MOVSD)
           | Build page tables:
           |   PML4 @ 0x1000
           |   PDPT @ 0x2000
           |   PD   @ 0x3000
           | Set CR3 = 0x1000 (PML4)
           | Set CR4.PAE = 1
           | Set EFER.LME = 1 (MSR 0xC0000080)
           | Set CR0.PG = 1
           | Far JMP selector 24
           v
+---------------------+
| 64-bit Long Mode    |  <-- target mode
| (target)            |
+----------+----------+
           |
           | Set RSP = 0x70000
           | JMP kernel_entry
           v
      [ kernel runs ]
```

FIG. 10 -- H-IR Instruction Set (37 opcodes)

==

#	Mnemonic	Format	Description
	ARITHMETIC		
1	IR_CONST	dst, imm	Load constant
2	IR_ADD	dst, src1, src2	Addition
3	IR_SUB	dst, src1, src2	Subtraction
4	IR_MUL	dst, src1, src2	Multiplication
5	IR_DIV	dst, src1, src2	Division
6	IR_MOD	dst, src1, src2	Modulo
7	IR_NEG	dst, src	Negate

```
+-----+------------+----------------+------------------------+
|     | BITWISE                                              |
+-----+------------+----------------+------------------------+
|  8  | IR_AND     | dst, src1, src2 | Bitwise AND           |
|  9  | IR_OR      | dst, src1, src2 | Bitwise OR            |
| 10  | IR_XOR     | dst, src1, src2 | Bitwise XOR           |
| 11  | IR_SHL     | dst, src1, src2 | Shift left            |
| 12  | IR_SHR     | dst, src1, src2 | Shift right           |
| 13  | IR_NOT     | dst, src        | Bitwise NOT           |
+-----+------------+----------------+------------------------+
|     | COMPARISON                                           |
+-----+------------+----------------+------------------------+
| 14  | IR_EQ      | dst, src1, src2 | Equal                 |
| 15  | IR_NE      | dst, src1, src2 | Not equal             |
| 16  | IR_LT      | dst, src1, src2 | Less than             |
| 17  | IR_LE      | dst, src1, src2 | Less or equal         |
| 18  | IR_GT      | dst, src1, src2 | Greater than          |
| 19  | IR_GE      | dst, src1, src2 | Greater or equal      |
+-----+------------+----------------+------------------------+
|     | CONTROL FLOW                                         |
+-----+------------+----------------+------------------------+
| 20  | IR_JMP     | label          | Unconditional jump     |
| 21  | IR_JZ      | src, label     | Jump if zero           |
| 22  | IR_JNZ     | src, label     | Jump if not zero       |
| 23  | IR_LABEL   | id             | Label definition       |
+-----+------------+----------------+------------------------+
|     | FUNCTION                                             |
+-----+------------+----------------+------------------------+
| 24  | IR_CALL    | dst, fn, args  | Function call          |
| 25  | IR_RET     | src            | Return value           |
| 26  | IR_PARAM   | dst, index     | Get parameter          |
+-----+------------+----------------+------------------------+
|     | MEMORY                                               |
+-----+------------+----------------+------------------------+
| 27  | IR_LOAD    | dst, addr      | Load from memory       |
| 28  | IR_STORE   | addr, src      | Store to memory        |
| 29  | IR_ALLOCA  | dst, size      | Stack allocation       |
+-----+------------+----------------+------------------------+
|     | SSA                                                  |
+-----+------------+----------------+------------------------+
| 30  | IR_PHI     | dst, src1, src2 | SSA phi function      |
| 31  | IR_MOVE    | dst, src        | Register move         |
+-----+------------+----------------+------------------------+
|     | EXTENDED                                             |
+-----+------------+----------------+------------------------+
| 32  | IR_SYSCALL | dst, nr, args  | System call            |
```

```
| 33  | IR_LEA     | dst, base, off  | Load effective address  |
| 34  | IR_ZERO    | dst             | Zero register           |
| 35  | IR_SIGN_EX | dst, src        | Sign extend             |
| 36  | IR_CMOV    | dst, cond, src  | Conditional move        |
| 37  | IR_MEMCPY  | dst, src, len   | Memory copy             |
+-----+------------+-----------------+-------------------------+
```

```
FIG. 11 -- Register Allocation Strategy
================================================================

  14 General Purpose Registers (allocatable):
  +-------+---------------------------------------------------------+
  | RAX   |=====[live range A]==================                 |
  | RCX   |        ======[live range B]========                  |
  | RDX   |==[C]==        =======[D]====                         |
  | RBX   |                =====[E]===========================   |
  | RSI   |=====[F]===                                           |
  | RDI   |         =====[G]====                                 |
  | R8    |                      =====[H]========                |
  | R9    |==[I]==                                               |
  | R10   |         =====[J]============                         |
  | R11   |                           =====[K]====               |
  | R12   |                                      ===[L]===       |
  | R13   |=====[M]==============================                |
  | R14   |                 =====[N]====                         |
  | R15   |                                =====[O]====          |
  +-------+---------------------------------------------------------+

  Reserved (NOT allocatable):
  +-------+---------------------------------------------------------+
  | RSP   |XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX | Stack ptr
  | RBP   |XXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXXX | Frame ptr
  +-------+---------------------------------------------------------+

  Spill to stack:
  +----------------+
  | [RBP - 8 ] var1|
  | [RBP - 16] var2|  <-- "Spill to stack" for exceeded ranges
  | [RBP - 24] var3|
  +----------------+

  System V AMD64 ABI calling convention:
  +------+------+------+------+------+------+--------+
  | arg0 | arg1 | arg2 | arg3 | arg4 | arg5 | return |
  | RDI  | RSI  | RDX  | RCX  | R8   | R9   | RAX    |
```

```
+------+------+------+------+------+------+--------+
```

FIG. 12 -- OS Toolchain Dependency Comparison

```
================================================================

+----------+----------+----------+----------+----------+---------+------+
| OS       | Kernel   | Compiler | Assembler| Linker   | Image    | Ext  |
|          | Language |          |          |          | Tool     | Deps |
+----------+----------+----------+----------+----------+---------+------+
| Linux    | C + asm  | GCC (C)  | gas (C)  | ld (C)   | make+dd  |  4+  |
+----------+----------+----------+----------+----------+---------+------+
| Redox    | Rust     | rustc    | LLVM     | lld      | cargo    |  3+  |
|          |          | (Rust)   | (C++)    |          |          |      |
+----------+----------+----------+----------+----------+---------+------+
| TempleOS | HolyC    | HolyC    | built-in | built-in | built-in |  1   |
|          | + asm    | + asm    |          |          |          | (asm)|
+----------+----------+----------+----------+----------+---------+------+
| Oberon   | Oberon   | Oberon   | built-in | built-in | external |  1   |
+----------+----------+----------+----------+----------+---------+------+
|*HicOS*   |*H-L*     |*H-L*     |*H-L*     |*H-L*     |*H-L*     |* 0* |
+----------+----------+----------+----------+----------+---------+------+
                                                          ^^^^^^^^
                                                    HicOS row: ZERO
                                                    external dependencies
```